

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms

SECTION: CPE21S1

DATE PERFORMED: Sep 10, 2026

DATE SUBMITTED: Sep 15, 2026

NAME: Mendoza, Roy Gabriel I.

INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 8.1

Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

A.1. Shell Sort

General Algorithm

The general algorithm for shell sort is given below.

```
shellSort(array, size)
  for interval i <- size/2n down to 1
    for each interval "i" in array
      sort all the elements at interval "i"
  end shellSort
```

A.2. Merge Sort

General Algorithm

The general pseudo-code for the merge sort technique is given below.

```
Declare an array Arr of length N
If N=1, Arr is already sorted
If N>1,
  Left = 0, right = N-1
  Find middle = (left + right)/2
  Call merge_sort(Arr, left, middle) -> sort first half recursively
  Call merge_sort(Arr, middle+1, right) -> sort second half recursively
  Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
Exit
```

A.3. Quick Sort

```
quicksort(A, low, high)
begin
  Declare array A[N] to be sorted
  low = 1st element; high = last element; pivot
  if(low < high)
    begin
      pivot = partition (A, low, high);
      quicksort(A, low, pivot-1)
      quicksort(A, pivot+1, high)
    End
  End
end
```

Code + Console Screenshot	<pre> 10 const int max_size = 20; 11 12 → void showArr(const int arr[], int size); 13 → void resetArr(const int src[], int dest[], int size); 14 15 int main() { 16 int original[max_size]; 17 int arr[max_size]; 18 19 // Seed the random number generator 20 srand(Seed:time(0)); 21 22 // Generate random values once for the original array 23 for (int i = 0; i < max_size; i++) { 24 original[i] = (rand() % 10)+1; 25 } 26 27 // Display original unsorted array 28 std::cout << "===== ORIGINAL ARRAY =====\n"; 29 showArr(original, max_size); 30 std::cout << "\n"; </pre> "C:\Users\royga\CLionProjects\untitled1\H0A 8p2\Main .exe" <pre> ===== ORIGINAL ARRAY ===== 7 9 10 3 3 5 7 1 5 1 3 9 3 3 7 9 3 8 3 10 </pre>
Observations	The main driver initializes an array with 20 elements and make the elements be between 1 and 10 using the (rand()%10)+1. And in terms of the time complexity, it is an O(n) because it need to populate the entire array for an n amount of time depending to the maxsize that we have.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	<pre> 9 //===== 10 //shell sort 11 template <typename T> 12 void shellSort(T arr[], int n) { 13 // Start with a big gap, then reduce the gap 14 for (int gap = n / 2; gap > 0; gap /= 2) { 15 16 // Do a gapped insertion sort for this gap size. 17 // The first gap elements arr[0..gap-1] are already 18 // in gapped order, so we start from arr[gap]. 19 for (int i = gap; i < n; i++) { 20 T temp = arr[i]; // element to be positioned 21 int j = i; 22 23 // Shift earlier gap-sorted elements up until the 24 // correct location for arr[i] is found 25 while (j >= gap && arr[j - gap] > temp) { 26 arr[j] = arr[j - gap]; 27 j -= gap; 28 } 29 30 // Put temp (the original arr[i]) in its correct location 31 arr[j] = temp; 32 } 33 } 34 35 // --- SHELL SORT --- 36 std::cout << "===== SHELL SORT =====\n"; 37 resetArr(original, arr, max_size); 38 shellSort(arr, n: max_size); 39 showArr(arr, max_size); 40 std::cout << "\n"; 41 42 ===== SHELL SORT ===== 43 1 1 3 3 3 3 3 3 3 5 5 7 7 7 8 9 9 9 10 10 </pre>
Observations	Shell sort optimizes the insertion sort by comparing and swapping the non adjacent elements across a gap that is getting smaller. The process begins with an initial gap of 10, since it is $n/2$ and out n is 20. This multi pass reduction allows distant out of order elements to make large jumps across the array early on, significantly decreasing the total shift operations required during the final insertion pass where the gap is finally equal to 1. And in terms of time complexity, the best case is $O(n \log n)$ and the worst is $O(n^2)$.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
34 //=====
35 //merge sort
36 template <typename T>
37 void merge(T arr[], int left, int middle,
38           int right, int size, T* temp) { // number
39     T* temp = new T[size]; // heap-alloc
40
41     int i = left; // i walks t
42     int j = middle + 1; // j walks t
43     int temp_pos = 0; // temp_pos
44
45     while (i <= middle && j <= right) { //
46         if (arr[i] < arr[j]) { //
47             temp[temp_pos] = arr[i]; //
48             i++; //
49             temp_pos++; //
50         } else { //
51             temp[temp_pos] = arr[j]; //
52             j++; //
53             temp_pos++; //
54         }
55     }while (i <= middle) { //
56         temp[temp_pos] = arr[i]; //
57         i++; //
58         temp_pos++; //
59     }while (j <= right) { //
60         temp[temp_pos] = arr[j]; //
61         j++; //
62         temp_pos++; //
63     }
64     int k = 0; //
65     for (int i = left; i <= right; i++) { //
66         arr[i] = temp[k]; //
```

	<pre> 66 arr[i] = temp[k]; // copy 67 k++; // advance 68 } 69 delete[] temp; // free 70 } 71 72 template <typename T> 73 void mergeSort(T arr[], int left, int right) { 74 if (left >= right) { 75 return; // base case 76 } 77 78 int middle = (left + right) / 2; 79 80 mergeSort(arr, left, middle); 81 mergeSort(arr, middle + 1, right); 82 merge(arr, left, middle, right); 83 } </pre> <pre> 39 // --- MERGE SORT --- 40 std::cout << "===== MERGE SORT =====\n"; 41 resetArr(original, arr, max_size); 42 mergeSort(arr, left: 0, right: max_size - 1); 43 showArr(arr, max_size); 44 std::cout << "\n"; </pre> <pre> ===== MERGE SORT ===== 1 1 3 3 3 3 3 3 5 5 7 7 7 8 9 9 9 10 10 </pre>
Observations	Merge sort implements a recursive divide and conquer strategy to sort the dataset. The mergeSort function continuously splits the array into a single element sub arrays by calculating the midpoint. Then, we have the merge function to compare elements across the adjacent sub arrays, and basically bubble sorting them. In terms of time complexity, it offers $O(n \log n)$.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```
85 //quicksort
86 template <typename T>
87 int partition(T arr[], int left, int right) {
88     T pivot = arr[right];           // cho
89     int low = left - 1;             // 'low' s
90     int high = right;               // 'high'
91
92     while (true) {
93         do{
94             low++;                    //
95         } while(arr[low] < pivot);    //
96         do {
97             high--;                  //
98         } while (high > left && arr[high] > pi
99         if (low >= high) {
100             break;
101         }
102         T tempSwap = arr[low];
103         arr[low] = arr[high];
104         arr[high] = tempSwap;
105     }
106     T tempPivot = arr[low];
107     arr[low] = arr[right];
108     arr[right] = tempPivot;
109     return low;
110 }
```

	<pre> 112 // Recursively sorts arr[left..right] using the q 113 template <typename T> 114 void quickSort(T arr[], int left, int right) { 115 if (left >= right) { 116 return; 117 } 118 int pivotIndex = partition(arr, left, right); 119 quickSort(arr, left, right: pivotIndex - 1); 120 quickSort(arr, left: pivotIndex + 1, right); 121 } 122 123 // Overload so callers can write quickSort(arr, n 124 template <typename T> 125 void quickSort(T arr[], int n) { 126 quickSort(arr, left: 0, right: n - 1); 127 } 128 #endif //UNTITLED1_SORTALGO_H </pre> <pre> 47 // --- QUICK SORT --- 48 std::cout << "===== QUICK SORT =====\n"; 49 resetArr(original, arr, max_size); 50 quickSort(arr, n: max_size); 51 showArr(arr, max_size); 52 std::cout << "\n"; </pre> <pre> ===== QUICK SORT ===== 1 1 3 3 3 3 3 3 3 5 5 7 7 7 8 9 9 9 10 10 </pre>
Observations	Quick sort also uses the divide and conquer approach. The partition function selects the rightmost element as the pivot, then uses the low and high pointers to scan inward, swapping elements smaller than the pivot. The pivot is then places into its final position at index low, allowing quicksort to recursively sort the left and right sub arrays. And in terms of time complexity, it is $O(n \log n)$ when partitions divide the array evenly, but can be $O(n^2)$ if is is always an imbalance split.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

7. Supplementary Activity

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have $O(N \cdot \log N)$ for their time complexity?

PROBLEM 1:	From my understanding, yes it can. Common hybrid strategy like the introsort, which switches to algorithms like insertion sort for small partitions to reduce the execution overhead. Example, an unsorted array of [48, 34, 29, 12, 53, 4], selecting 12 as the pivot partitions the data into a left sublist of 4, the pivot, then the rest. So instead of quick sort recursively on the right sub list, insertion sort can process the right side by shifting the elements in place to produce the [29, 34, 48, 53]. Source: https://www.programiz.com/dsa/sorting-algorithm
PROBLEM 2:	Using the provided dataset, quick sort delivers the fastest real world performance, with insertion sort acting as a close alternative due to the very small array size. Quick sort runs faster because it sorts in place, eliminating the overhead of allocating temporary memory buffers while taking advantage of cpu caches efficiency. Both algorithms achieve an overall of $O(n \log n)$ complexity through their divide and conquer principle. So they are almost the same in terms of time complexity, but still differ on the speed by the amount of the array. Source; https://medium.com/@francoiskabore422/the-top-3-sorting-algorithms-a-performance-showdown-4eb69a7bb72f

CONCLUSION:

Through this activity, we have learned the fundamentals of the sorting algorithms including their time complexity. We have been able to demonstrate how shell sort optimizes the insertion sort through decreasing the gap intervals, while the divide and conquer approach of both the merge and quick sort reduce the average execution time to $O(n \log n)$. And in the supplementary activity, we have explored or just see ahead about the hybrid sorting algorithms, like applying insertion sort to quick sort partitions.

And as for my own personal analysis of what I have done in this activity, I would like to thank my friends and programiz to further explain the code using the code itself, not just the pseudocode itself. Then what I can see myself in need of improvement, would be the application of the pseudocode itself, and analyzing it faster, because this has become like 2 days per code, not the shell sort, but both the merge and quick sort.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.